Week 4 Task

Machine Learning Model Selection and Evaluation Plan

Project: Road Accident Severity Prediction

Dataset: US Accidents (Kaggle) | Virtual Data Science Apprentice – Python Specialist Intern

Executive Summary

This document presents the model selection and evaluation strategy for the fourth and final phase of the Road Accident Severity Prediction project. Building on the data cleaning plan (Week 2) and the exploratory data analysis strategy (Week 3), this plan frames severity prediction as a multi-class classification problem, shortlists five candidate algorithms, defines the evaluation metrics that matter most given the safety context of the problem, and lays out a concrete validation and hyperparameter-tuning procedure. Two of the recurring themes from prior feedback — the need for worked numerical examples and explicit handling of rare categorical values such as uncommon weather types — are addressed directly in Sections 4 and 6.

1. Introduction: The Landscape of Machine Learning Models

A machine learning model is only as good as its fit to the structure of the problem, the data available, and the operational constraints under which it will be used. For the US Accidents dataset, the target variable is accident Severity, typically encoded as an ordinal/categorical scale (e.g., 1 = Minor, 2 = Moderate, 3 = Serious, 4 = Fatal in the raw data, which this plan collapses into three practical classes: Minor, Serious, and Fatal, to reduce sparsity in the rarest class). This makes the task a multi-class classification problem, and in some formulations an ordinal classification problem, since severity levels have a natural order.

Broadly, the models available fall into four families:

- Linear / statistical models — e.g., Logistic Regression, Ordinal Logistic Regression. Fast, interpretable, and a strong baseline, but limited in capturing non-linear interactions (e.g., the combined effect of rain + poor visibility + high speed limit).
- Tree-based models — e.g., Decision Trees, Random Forests, Gradient Boosted Trees (XGBoost, LightGBM). Handle non-linearity and mixed categorical/numerical features well; robust to unscaled data; provide native feature importance.
- Distance / margin-based models — e.g., k-Nearest Neighbors, Support Vector Machines. Can work well on well-scaled, lower-dimensional data but scale poorly to the size of this dataset (millions of rows) and are sensitive to the curse of dimensionality after one-hot encoding weather/road-condition categories.
- Neural networks — e.g., a Multi-Layer Perceptron (MLP). Capable of learning complex feature interactions automatically, but require more data preprocessing, more tuning, and sacrifice interpretability.

Four factors drive the choice of model for this specific project:

- Class imbalance: Fatal accidents are a small minority of records (commonly well under 5% in the US Accidents dataset), so the chosen model and evaluation approach must not simply reward predicting the majority class.
- Mixed feature types: The dataset combines numerical fields (temperature, visibility, wind speed), categorical fields (weather condition, road feature flags, time of day), and geospatial fields (latitude/longitude, state) — tree ensembles handle this mix more gracefully than linear models.
- Interpretability requirements: Because the outcome relates to road safety, stakeholders (transport authorities, insurers) will expect to understand which factors drive a “Fatal” prediction, not just receive a black-box score.
- Computational budget: The full dataset has millions of rows; training time and memory footprint influence whether a full grid search or a more efficient randomized/Bayesian search is feasible.

2. Problem Framing and Success Criteria

2.1 Target Variable

Severity will be modeled as a 3-class target: Minor, Serious, Fatal. Collapsing the original 4-level scale into 3 classes is a direct response to the categorical-sparsity concern raised in the Week 3 review (rare weather types compound with an already-rare Fatal class, leaving very few training examples for combinations such as “Fatal + Sleet”).

2.2 Success Criteria

- Primary metric: Macro-averaged F1-score, so performance on the rare but high-stakes Fatal class is not masked by the abundant Minor class.
- Secondary constraint: Recall on the Fatal class ≥ 0.70, because a missed Fatal-severity prediction (false negative) is operationally more costly than a false alarm.
- Tertiary requirement: Model must produce feature importance or SHAP values usable in a stakeholder-facing report.

3. Model Selection Criteria

Each candidate model is scored qualitatively (High / Medium / Low) against six criteria before any code is run, so that the shortlist in Section 4 is defensible rather than arbitrary:

Criterion	Why it matters for this project
Handles mixed data types	Weather, road-feature flags, and numeric sensor readings coexist in every record.
Robust to class imbalance	Fatal accidents are a small fraction of ~3–5% of records.
Interpretability	Findings must be explainable to non-technical transportation stakeholders.
Training/inference speed at scale	US Accidents has several million rows; iteration speed affects how much tuning is feasible.
Ability to capture non-linear interactions	Severity likely depends on interactions, e.g., low visibility × high speed limit × night-time.
Availability of mature Python tooling	scikit-learn / XGBoost / LightGBM / imbalanced-learn ecosystem must support the chosen approach.

4. Candidate Machine Learning Approaches

Five candidates are carried forward for empirical comparison. Each subsection below states the model's mechanism in one or two sentences, its suitability score against Section 3's criteria, and — where useful — a small worked numerical illustration.

4.1 Logistic Regression (Baseline)

Logistic Regression estimates class probabilities via a linear combination of features passed through a softmax function for the multi-class (one-vs-rest or multinomial) case. It is included purely as an interpretable baseline against which more complex models must prove their added value.

Worked example: For a single accident record with standardized features Visibility_z = -1.2, SpeedLimit_z = 0.8, and IsNight = 1, and fitted coefficients β = [-0.9 (visibility), 1.1 (speed limit), 0.6 (is-night)] with intercept β0 = -2.0 for the Fatal-vs-rest logit:

$$logit = \beta 0 + \beta 1(-1.2) + \beta 2(0.8) + \beta 3(1) = -2.0 + 1.08 + 0.88 + 0.6 = 0.56$$

P(Fatal) = 1 / (1 + e^-0.56) ≈ 0.636. This single calculation illustrates how the model would flag this specific record as a higher-than-average Fatal risk (63.6% vs. an assumed base rate near 3–5%), which is the kind of case-level transparency stakeholders can audit.

Suitability: High interpretability and speed; Low on non-linear interaction capture; Medium on imbalance handling (mitigated with class_weight='balanced').

4.2 Decision Tree Classifier

A Decision Tree recursively splits the feature space on the variable and threshold that most reduces impurity (Gini or entropy) at each node. It naturally handles mixed data types and non-linear thresholds (e.g., "visibility < 2 miles") without scaling.

Suitability: High interpretability (can be visualized) and High for mixed data types; Medium speed; Low-Medium generalization (prone to overfitting without depth/leaf constraints) — used mainly as a stepping stone to Random Forests.

4.3 Random Forest Classifier

An ensemble of decision trees, each trained on a bootstrap sample of the data and a random subset of features at each split, with predictions aggregated by majority vote. This averaging reduces the variance (overfitting) of a single decision tree while retaining its ability to model non-linear interactions.

Suitability: High on mixed data types and non-linear interactions; Medium interpretability (via feature_importances_ and SHAP); Medium speed at several-million-row scale — mitigated by sampling or limiting max_depth/n_estimators during early experimentation.

4.4 Gradient Boosted Trees (XGBoost / LightGBM)

Boosting builds trees sequentially, where each new tree is fit to the residual errors (technically, the negative gradient of the loss) of the ensemble so far. LightGBM in particular uses histogram-based splitting and leaf-wise growth, making it well suited to the dataset's scale. Class weighting (scale_pos_weight or custom sample_weight) is well supported for imbalance.

Suitability: High across almost every criterion in Section 3 except raw interpretability, which is recovered via SHAP values. This is the leading candidate given the scale, mixed feature types, and class-imbalance profile of the dataset.

4.5 Multi-Layer Perceptron (Neural Network)

A feed-forward neural network with one or more hidden layers can, in principle, approximate any function mapping features to severity class probabilities. In practice, on tabular data of this kind, MLPs rarely outperform gradient-boosted trees and require careful scaling, more data to avoid overfitting, and offer the weakest native interpretability.

Suitability: Medium on non-linear interaction capture; Low on interpretability and training speed; included in the plan mainly as a stretch comparison, not the primary candidate.

4.6 Comparison Summary

Model	Handles Mixed Types	Imbalance Robustness	Interpretability	Speed @ Scale	Non-linear Capture
Logistic Regression	Medium	Medium*	High	High	Low
Decision Tree	High	Medium*	High	High	Medium
Random Forest	High	Medium*	Medium	Medium	High
Gradient Boosting (XGB/LGBM)	High	High*	Medium (w/ SHAP)	Medium-High	High

Model	Handles Mixed Types	Imbalance Robustness	Interpretability	Speed @ Scale	Non-linear Capture
Neural Network (MLP)	Medium	Medium*	Low	Low-Medium	High

** All models' imbalance robustness assumes class_weight / sample_weight or a resampling technique (Section 6.1) is applied; none of these models is imbalance-robust "out of the box."*

5. Evaluation Metrics and Rationale

Because the classes are imbalanced and the cost of errors is asymmetric (missing a Fatal case is worse than a false alarm), accuracy alone would be misleading — a model that always predicts "Minor" could reach roughly 90%+ accuracy while being clinically useless. The following metrics are therefore used together:

Metric	Formula	Why it is included
Accuracy	(TP+TN) / Total	Reported for context only; not used for model selection due to imbalance.
Precision (per class)	TP / (TP+FP)	Of all records predicted Fatal, how many truly are — controls false-alarm rate.
Recall (per class)	TP / (TP+FN)	Of all truly Fatal records, how many were caught — the priority metric for the Fatal class.
F1-score (per class)	2×(P×R)/(P+R)	Harmonic mean balancing precision and recall for each class.
Macro-F1	Mean of per-class F1	Primary model-selection metric; treats every class equally regardless of frequency.
Weighted-F1	Frequency-weighted F1	Reported alongside Macro-F1 to show overall performance including the majority class.
Confusion Matrix	n/a	Shows exactly which severities are confused with which — critical for error analysis.
ROC-AUC (One-vs-Rest)	Area under ROC curve	Threshold-independent view of class separability, computed per class.

5.1 Worked Example: Confusion Matrix and Metric Calculation

To make these formulas concrete, consider a hypothetical 1,000-record test set with the following confusion matrix (rows = actual, columns = predicted):

Actual \ Predicted	Minor	Serious	Fatal
Minor (n=800)	740	55	5
Serious (n=160)	40	104	16
Fatal (n=40)	3	9	28

From this matrix, the Fatal class metrics are calculated as follows:

- Precision (Fatal) = 28 / (5 + 16 + 28) = 28 / 49 ≈ 0.571
- Recall (Fatal) = 28 / (3 + 9 + 28) = 28 / 40 = 0.700 — meeting the ≥ 0.70 target set in Section 2.2
- F1 (Fatal) = 2 × (0.571 × 0.700) / (0.571 + 0.700) ≈ 0.629

Repeating the same calculation for Minor (Precision ≈ 0.945, Recall = 0.925, F1 ≈ 0.935) and Serious (Precision ≈ 0.619, Recall = 0.650, F1 ≈ 0.634) gives a Macro-F1 = (0.935 + 0.634 + 0.629) / 3 ≈ 0.733. This worked example demonstrates

exactly how a model that looks strong on Accuracy (740+104+28 = 872/1000 = 87.2%) can simultaneously reveal a real weakness on the Serious class (F1 ≈ 0.634), which the Macro-F1 metric surfaces but a bare accuracy figure would hide.

6. Anticipated Challenges and Mitigation Strategies

6.1 Class Imbalance

With Fatal accidents representing an estimated 3–5% of records, three complementary techniques will be evaluated: (a) class_weight='balanced' in scikit-learn / XGBoost's scale_pos_weight equivalent for multi-class via sample_weight, which penalizes misclassifying minority classes more heavily during training; (b) SMOTE (Synthetic Minority Over-sampling Technique, via the imbalanced-learn library) applied only to the training fold, never to validation/test data, to avoid information leakage; and (c) threshold adjustment on the predicted probabilities post-training, which is cheaper to iterate on than retraining.

6.2 Categorical Sparsity in Rare Weather Types

As flagged in the Week 3 review, weather conditions such as “Sleet,” “Squall,” or “Volcanic Ash” appear in a very small number of records. Three concrete strategies will be compared: (1) grouping rare categories (frequency below a 0.5% threshold) into a single “Other/Rare Weather” bucket before one-hot encoding, which is simple and prevents one-hot columns with near-zero variance; (2) target encoding (mean severity rate per category, computed with k-fold cross-fitting to avoid leakage), which preserves more signal than bucketing but requires careful validation; and (3) for tree-based models specifically, using native categorical support in LightGBM, which can split on categories directly without one-hot expansion and handles rare levels more gracefully than linear models.

6.3 Overfitting

Tree ensembles will use explicit regularization: max_depth limits, min_child_weight / min_samples_leaf minimums, subsample and colsample_bytree < 1.0 for boosting, and early_stopping_rounds monitored on a held-out validation fold so that boosting halts once validation Macro-F1 stops improving.

6.4 Interpretability of Ensemble Models

For the leading candidate (gradient boosting), SHAP (SHapley Additive exPlanations) values will be computed on a sample of the test set to produce both global feature-importance summaries and local, case-level explanations (e.g., “this record was flagged Fatal primarily due to low visibility and high speed limit”), directly answering the interpretability requirement from Section 2.2.

7. Model Testing, Validation, and Hyperparameter Tuning Strategy

7.1 Data Splitting

- Stratified split: 70% train / 15% validation / 15% test, stratified on Severity so the rare Fatal class is proportionally represented in every split.
- Test set is held out entirely until final model comparison (Section 8) — never touched during tuning.

7.2 Cross-Validation

Stratified K-Fold (k = 5) cross-validation will be used on the training set during hyperparameter search, again stratified on Severity, so that each fold preserves the same rare-class proportions rather than risking a fold with zero Fatal examples.

7.3 Hyperparameter Search Strategy

Given the dataset scale, a two-stage search is more computationally realistic than an exhaustive grid search:

- Stage 1 — RandomizedSearchCV across a wide parameter space (e.g., 40–60 random draws) to identify promising regions of the hyperparameter space cheaply.
- Stage 2 — GridSearchCV (or Bayesian optimization via Optuna, if time allows) narrowed around the best region found in Stage 1, for final fine-tuning.

Example parameter grid for the leading candidate (LightGBM):

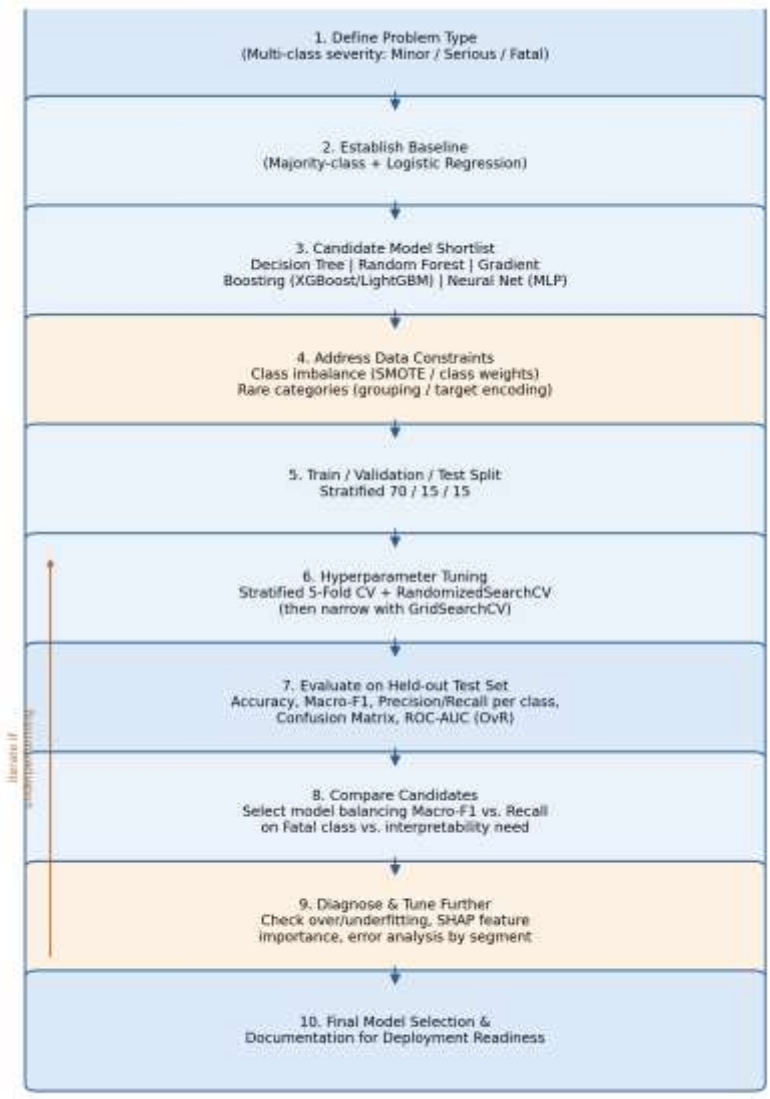
Hyperparameter	Search Range	Purpose
num_leaves	15, 31, 63, 127	Controls tree complexity / capacity.
learning_rate	0.01, 0.05, 0.1	Trade-off between convergence speed and generalization.
n_estimators	100–1000 (with early stopping)	Number of boosting rounds.
min_child_samples	10, 20, 50	Minimum data per leaf; guards against overfitting on sparse categories.
scale_pos_weight / class weights	1, 5, 10, computed ratio	Directly addresses class imbalance.
subsample / colsample_bytree	0.6–1.0	Row/column subsampling for regularization.

7.4 Model Comparison Protocol

All five candidates will be trained with their best-found hyperparameters and compared side by side on the untouched test set using the metric table from Section 5, plus training/inference time. The final choice will not be purely the highest Macro-F1 model; it will explicitly weigh the Section 2.2 constraint (Fatal recall ≥ 0.70) and interpretability needs, documenting any trade-off made.

8. End-to-End Process Flowchart

The diagram below visualizes the full pipeline from problem framing through final model selection, including the feedback loop back into hyperparameter tuning when a candidate underperforms against the Section 2.2 success criteria.



9. Recommendation and Phased Execution Plan

Based on the qualitative comparison in Section 4.6 and the constraints in Section 3, LightGBM (or XGBoost as a fallback if LightGBM's native categorical handling proves unstable on this dataset) is the recommended primary candidate, with Logistic Regression retained throughout as an interpretable baseline for sanity-checking results, and Random Forest as a secondary ensemble comparison. The execution plan is phased as follows:

- Phase 1 (Baseline): Train Logistic Regression and a shallow Decision Tree; record baseline Macro-F1 and per-class recall.
- Phase 2 (Ensemble candidates): Train Random Forest and LightGBM/XGBoost with class-imbalance handling from Section 6.1 and category treatment from Section 6.2.
- Phase 3 (Tuning): Run the two-stage hyperparameter search from Section 7.3 on the strongest ensemble candidate.
- Phase 4 (Stretch comparison): Train an MLP baseline for completeness, primarily to confirm that the added complexity is not justified by a Macro-F1 gain.
- Phase 5 (Final evaluation): Apply the comparison protocol from Section 7.4 on the held-out test set; generate confusion matrices, SHAP summaries, and a final model card documenting the chosen model, its metrics, and known limitations.

10. Python Tools and Libraries

Library	Role in this plan
scikit-learn	Logistic Regression, Decision Tree, Random Forest, train/test split, StratifiedKFold, GridSearchCV/RandomizedSearchCV, metrics.
xgboost / lightgbm	Gradient-boosted tree implementations with native class-weighting and (LightGBM) categorical support.
imbalanced-learn	SMOTE and related resampling techniques, applied within a cross-validation-safe Pipeline.
shap	Global and local model interpretability for the ensemble candidate.
optuna (optional)	Bayesian hyperparameter optimization as an alternative to Stage 2 grid search.
pandas / numpy	Data loading, feature engineering (carried over from Weeks 1–2).
matplotlib / seaborn	Confusion matrix heatmaps, ROC curves, and SHAP plots for the final report.

11. Conclusion

This plan translates the exploratory findings from Weeks 1–3 into a concrete, defensible modeling strategy: a shortlist of five candidates scored against explicit criteria, a metric set chosen specifically for an imbalanced, safety-relevant target, worked numerical examples showing exactly how those metrics behave, direct mitigations for the class-imbalance and rare-weather-category issues raised in earlier feedback, and a two-stage validation/tuning procedure sized to the dataset's scale. The result is a plan that could be handed to another team member to execute without further clarification, which was the explicit goal of this Week 4 deliverable.